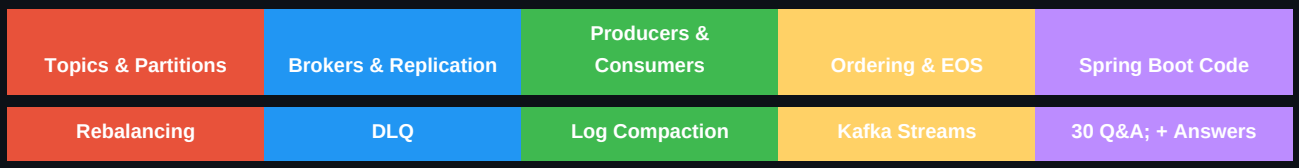
Apache Kafka

Complete Engineering & Interview Master Reference



Spring Boot Developer Edition · Zalando & FAANG Interview Preparation

Every WHY explained — Architecture to Exactly-Once Semantics

01 What Is Kafka? — Core Mental Model & Architecture
02 Topics, Partitions & Key-Based Routing
03 Brokers, Leaders, Followers, ISR & High Watermark
04 How a Message Travels: Full Producer Flow
05 The Append-Only Log: Why Kafka Is Fast
06 Producers Deep Dive: acks, Batching, Idempotency, Transactions
07 Consumers Deep Dive: Groups, Offsets, Auto vs Manual Commit
08 Threading Model & Timing Parameters
09 Ordering Guarantees — The Golden Rule
10 Consumer Group Rebalancing: Eager vs CooperativeSticky
11 Delivery Guarantees & Exactly-Once Semantics
12 Dead Letter Queue (DLT) — Spring Boot Patterns
13 Log Compaction — The Distributed KV Store Pattern
14 Kafka Streams — Real-Time Processing
15 Queue vs Stream: When to Use Which
16 Scalability, Capacity Numbers & Hot Partitions
17 Performance Optimisations
18 Complete Configuration Reference
19 30 Hard Interview Q&A; — Zalando / FAANG Level
20 Master Cheat Sheets

01 What Is Kafka? — Core Mental Model & Architecture

Kafka is a **distributed, partitioned, replicated commit log service**. Originally built at LinkedIn (2010), open-sourced via Apache (2011). Used by 80% of the Fortune 100. The core abstraction is an append-only, ordered, immutable log of events.

■ CRITICAL Mental Model

- Kafka is NOT a message queue. It is a distributed log.
- Messages are NOT deleted on consumption — they persist based on retention policy.
- Multiple consumer groups can independently replay the same data at any time.
- This single insight explains every design decision Kafka makes.

Four Core Pillars

Pillar	What it means	Why it matters
Distributed	Runs as a cluster of broker servers across machines	No single point of failure; scales horizontally
Partitioned	Each topic split into parallel ordered sub-logs	Unit of parallelism — more partitions = more consumers = more throughput
Replicated	Each partition copied across multiple brokers	Data survives broker failures without loss
Commit Log	Append-only, offset-indexed; retained by time/size policy	Immutability enables replay, replication, and sequential I/O speed

Kafka vs Traditional Message Queue

Aspect	Traditional Queue	Kafka
Message deleted after consume?	Yes — gone once read	No — persists by retention policy
Multiple consumers same data?	No — one consumer per message	Yes — unlimited consumer groups
Ordering guarantee?	Per queue (single)	Per partition (per key)
Replay old messages?	Not possible	Yes — reset offsets anytime
Throughput	Moderate	Millions/sec per partition
Persistence	Usually in-memory	Durable disk storage + replication

"Kafka is a distributed log. Everything else — queuing, streaming, replication, scaling — flows from that single idea."

02 Topics, Partitions & Key-Based Routing

What Is a Topic?

A topic is a named category to which producers publish messages. Topics are **multi-subscriber** — many consumer groups can independently read the same topic. Topics are NOT queues — data persists even after being consumed. Convention: kebab-case describing the event (e.g. *order-placed*, *payment-done*).

What Is a Partition?

A partition is an **ordered, immutable sequence of records** — a sub-log. Records are only ever appended — no updates, no deletes mid-stream. Each record gets a unique monotonically increasing **offset**. Partitions are stored on disk as segment files (.log, .index, .timeindex). One consumer per partition per consumer group — this enforces ordering.

Why Both Topics AND Partitions?

A topic is **logical** — just a name you publish to. A partition is **physical** — an actual file on disk on a specific broker. Topics give you a clean API. Partitions give you parallelism and ordering. Without partitions: single file, single machine, no scale. Without topics: no way to separate soccer events from basketball events.

Key-Based Partition Routing — The Ordering Mechanism

```
# Partition selection algorithm (murmur2 hash by default)

targetPartition = hash(key) % numberOfPartitions


# Example: key = 'CUST-101'

hash('CUST-101') = 2847392

2847392 % 6 partitions = 4

→ ALL messages for CUST-101 ALWAYS go to Partition 4


# Same key = same partition = same consumer thread = guaranteed order
```

■ KEY INSIGHT — Why the Key Is Everything

- Same key → same partition → same consumer thread → guaranteed order.
- This is how Kafka guarantees ORDER_PLACED → PAYMENT_DONE → SHIPPED → DELIVERED per customer.
- If SHIPPED arrives before PAYMENT_DONE, your state machine breaks. The key prevents this.

• Without a key: Kafka uses sticky partitioner (batches to one partition, then rotates) — throughput but no ordering.

Zalando: Two Topic Design Patterns

Design 1 — Event Envelope (One Topic, Multiple Event Types)

Use when you need strict ordering across event types for the same entity.

```
Topic: 'customer-order-events' Key: customer_id

Value: { eventType: ORDER_PLACED | PAYMENT_DONE | SHIPPED | DELIVERED, payload: {...} }


customer-order-events:

Partition 0: CUST-200 [ORDER_PLACED → PAYMENT_DONE → SHIPPED]

Partition 4: CUST-101 [ORDER_PLACED → PAYMENT_DONE → ITEM_PACKED → SHIPPED → DELIVERED]


// One consumer thread owns partition 4 — sees CUST-101 events in GUARANTEED ORDER

kafkaTemplate.send('customer-order-events', customerId, event);
```

Design 2 — Separate Topics Per Event Type

Use when different teams own different stages, need independent scaling, and cross-event ordering is NOT required. **Warning:** cross-topic ordering is never guaranteed.

```
Topic: 'order-placed' Key: order_id

Topic: 'payment-done' Key: order_id

Topic: 'order-shipped' Key: order_id

// Different teams consume different topics independently

// CANNOT guarantee ORDER_PLACED happens before PAYMENT_DONE across topics
```

Partition Count Rules

Rule	Detail
More partitions = more parallelism	More consumer instances can run simultaneously; higher throughput ceiling
More partitions = more overhead	More file handles, more replication network traffic, more memory
Cannot reduce count after creation	Only increase — key-ordering BREAKS on increase (hash % N changes)
Partition count >= max expected consumers	Extra consumers sit idle — plan for Black Friday peak, not average
Over-partition slightly upfront	Adding partitions later breaks key routing; changing it is painful

"A topic is a label. A partition is a file. The key determines which file your message goes to — and therefore its ordering."

03 Brokers, Leaders, Followers, ISR & High Watermark

What Is a Broker?

A broker is a single Kafka server process (JVM). Each broker gets a unique numeric ID. A broker stores partition data on local disk and serves producers and consumers. One broker per cluster is elected **Controller Broker** — it manages partition leadership changes. ZooKeeper (or KRaft in Kafka 3.3+) stores cluster state and coordinates.

■ CRITICAL INSIGHT — Leadership is Per-Partition

- A broker does NOT 'become a leader'. Specific PARTITIONS hosted on a broker are elected leaders.
- One broker can be leader for 20 partitions and follower for 40 others SIMULTANEOUSLY.
- Every broker is simultaneously LEADER for some partitions and FOLLOWER for others.

Concrete Broker Distribution — 3 Brokers, 2 Topics, RF=3

```
■■ BROKER 1 (10.0.0.1:9092) ■■■
```

Accepts ALL writes from producers	Continuously polls leader with FetchRequests
Maintains the authoritative log	Appends fetched records to its own replica
Tracks ISR — which followers are caught up	Reports its Log End Offset (LEO) to leader
Advances the High Watermark (HW)	Included in ISR if within replica.lag.time.max.ms
Serves consumer fetch requests (up to HW)	Does NOT serve reads or writes — standby only
Serves follower fetch requests	Becomes leader candidate if current leader fails

ISR, LEO & High Watermark — Concrete Example

```
Broker-1 [LEADER] offsets 0-9 written LEO = 10

Broker-2 [ISR] offsets 0-9 written LEO = 10

Broker-3 [FOLLOWER] offsets 0-7 written LEO = 8 (lagging — GC pause)


ISR = [Broker-1, Broker-2] (Broker-3 removed from ISR — too slow)

HW (High Watermark) = min(ISR LEOs) = min(10, 10) = 10


Consumers can see up to offset 9 (HW-1)

WHY? Prevents reads of data that might be rolled back if leader fails.


Broker-3 keeps fetching → rejoins ISR once its LEO catches up to 10
```

Leader Election — Step by Step

```
BEFORE: Broker-1 leads order-placed/P0, ISR=[B1,B2,B3], HW=5


Step 1: Broker-1 crashes (OOM / hardware failure)

Step 2: ZooKeeper ephemeral node for Broker-1 disappears

Step 3: Controller notices heartbeat gone (session.timeout elapsed)

Step 4: Controller picks new leader from ISR: [B2, B3] → picks B2

Step 5: New assignment written to ZooKeeper / KRaft metadata log

Step 6: All brokers notified; producers/consumers fetch new metadata

Step 7: Producers/consumers reconnect to B2 — no application code change

Step 8: Broker-1 restarts → truncates log to HW=5 → fetches from B2
```

```
Broker-1 rejoins ISR after catching up
```

■■ unclean.leader.election — Know This Setting

- Default: false — election ONLY from ISR = ZERO data loss.
- If true: out-of-sync replicas may become leader = POSSIBLE DATA LOSS.
- Only use true if availability > durability (e.g. metrics topic, logs — NOT payments).

Production Durability Configuration

```
replication.factor = 3 # survive 2 broker failures

min.insync.replicas = 2 # at least 2 replicas must confirm write

acks = all # producer waits for all ISR acks


A write is confirmed only when 2 replicas have it.

Even if the leader crashes AFTER ack: follower in ISR has the data → NO LOSS.

One broker can be completely lost with ZERO data loss.
```

"Leadership is per-partition. ISR = replicas that are fully caught up. HW = what all ISR members have confirmed. Consumers only read up to HW."

04 How a Message Travels: Full Producer Flow

Knowing the end-to-end path lets you answer almost any 'what happens when X fails?' question.

Step	Stage	What happens & WHY
1	**Message Created**	Producer builds: value (payload), key, timestamp, headers. All optional but key is critical for ordering.
2	**Partition Selected**	hash(key) % num_partitions. Same key always → same partition. No key → sticky partitioner: batch to one, then rotate.
3	**Broker Lookup**	Producer holds local metadata map (partition → broker address). Connects directly to leader broker. No middleman.
4	**Written to Leader**	Broker appends message to END of the partition log. Sequential write — no disk seek. Extremely fast.
5	**Replicated to Followers**	Followers fetch new entries from leader and append. When all ISR members confirm, leader sends ack to producer.
6	**Consumer Polls**	Pull model — consumers ask broker for new messages at their own pace. Broker returns batch up to HW.
7	**Offset Committed**	Consumer writes position to __consumer_offsets. On restart, resumes from saved offset — at-least-once delivery.

■■ Metadata Staleness Handling

- If broker dies after producer cached its metadata, next send fails.
- Producer automatically refreshes metadata from the controller and retries — transparent.
- Causes a brief latency spike but no data loss if acks=all is configured.

"Producer hashes key → looks up broker in local metadata → writes directly. No gateway, no routing layer."

05 The Append-Only Log: Why Kafka Is Fast

The Physical Reality of Disks

A spinning hard drive has a physical read/write head that moves across a magnetic platter. Moving that head takes real time — this is called a **seek**. Seeks are the dominant bottleneck. Even SSDs benefit enormously from sequential patterns due to how NAND flash and OS page caching work.

```
RANDOM WRITES (traditional DB): SEQUENTIAL WRITES (Kafka):

msg → seek to pos 847 ← time wasted msg → end of log ← write

msg → seek to pos 12 ← time wasted msg → end of log ← write

msg → seek to pos 3201 ← time wasted msg → end of log ← write

msg → end of log ← write

Result: ~100 writes/sec (HDD) Result: ~1M writes/sec
```

Three Benefits That Cascade From Append-Only

Immutability

Once written, messages cannot be edited or deleted individually. This makes replication trivially simple: a follower just asks 'give me everything after offset N' and appends it. No conflict resolution, no merging concurrent edits, no distributed locks. Also simplifies recovery — on restart, replay from checkpoint.

Efficiency

Sequential I/O is ~100x faster than random I/O on spinning drives, significantly faster even on SSDs. Kafka also uses ZERO-COPY transfer: the OS moves data from disk to network socket without ever entering userspace. This is why a single broker can handle 1M messages/sec on commodity hardware.

Scalability

Easy to split (more partitions = more independent logs), easy to replicate (stream the log tail), easy to clean up (delete whole old segment FILES — not individual records). No coordination between partitions needed — each is fully independent.

■ Why Kafka Never Deletes Individual Messages

- Deleting mid-log requires a random write — destroying the sequential advantage.
- Instead, Kafka deletes entire old segment FILES from the start of the log.
- This is O(1): delete a file. Not O(n): scan and remove individual records.

"A disk write is fast. A disk seek is slow. Append-only means you seek once — to the end — then just keep writing."

06 Producers Deep Dive: acks, Batching, Idempotency, Transactions

Acknowledgment Modes (acks)

acks	Behaviour	Data Loss Risk	Throughput	Use Case
0	No wait — fire and forget	HIGH — message may vanish	~1M/s	Clickstream, hover events, metrics
1	Leader writes & ACKs. Followers async.	MEDIUM — leader crash loses msg	High	Log aggregation, audit trails
all / -1	ALL ISR replicas confirm. Then ACK.	ZERO (with min.insync=2)	Moderate	Orders, payments, inventory

acks=0 — Clickstream (Fire & Forget)

```
config.put(ProducerConfig.ACKS_CONFIG, '0');

config.put(ProducerConfig.LINGER_MS_CONFIG, 5);

config.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, 'lz4');

// Does NOT wait even for OS to confirm receipt

// If Broker crashes 1ms later: message LOST. Producer never knows.

// Use case: user clicked a product page — losing 0.01% clicks doesn't matter
```

acks=1 — The Hidden Risk

```
config.put(ProducerConfig.ACKS_CONFIG, '1'); // leader only


Timeline:

Producer → Broker-1 (leader of P0)

Broker-1 writes to local disk → sends ACK ← producer gets this

Broker-2 and Broker-3 replicate AFTER the ACK is sent

RISK: Broker-1 crashes BETWEEN ACK and follower replication

→ message acknowledged to producer but LOST PERMANENTLY
```

acks=all — Order / Payment Events (Zero Loss)

```
config.put(ProducerConfig.ACKS_CONFIG, 'all');

config.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);

config.put(ProducerConfig.RETRIES_CONFIG, Integer.MAX_VALUE);


T+0ms: Producer sends to Broker-1 (leader of order-placed/P0)

T+2ms: Broker-1 writes to local log

T+4ms: Broker-2 replicates + ACKs to Broker-1

T+4ms: min.insync.replicas=2 satisfied → Broker-1 ACKs producer

Even if Broker-1 NOW crashes → Broker-2 has the data → NO LOSS
```

Batching, linger.ms & Compression

```
WITHOUT BATCHING (linger.ms=0):

1000 price updates → 1000 separate network calls

Each call: ~1ms network overhead → 1000ms just in network


WITH BATCHING (linger.ms=20, batch.size=64KB):

T+0ms: msg-1 arrives → sits in buffer

T+20ms: linger.ms fires → send all accumulated

OR: buffer hits 64KB → send immediately (whichever first)

Result: 1 network call carrying 50 messages → ~50x improvement


COMPRESSION (snappy on JSON batches):

50 JSON price updates × 200 bytes = 10,000 bytes raw

snappy → ~2,500 bytes (75% reduction)

Network sends 2.5KB instead of 10KB
```

Algorithm	Ratio	CPU Cost	Best For
none	0%	Zero	Low volume, latency-critical
lz4	~40%	Very Low	High frequency, small messages, latency matters
snappy	~60%	Low	General purpose — Zalando default for order events

gzip	~70%	High (5× snappy)	Archival, batch pipelines
zstd	~75%	Moderate	Best ratio+speed — ideal for high-volume analytics

Idempotent Producer — Preventing Message Reordering

```
PROBLEM — Without idempotency (max.in.flight=5, retries=3):

5 messages in flight: [msg1, msg2, msg3, msg4, msg5]

Network glitch on msg1 only → msg2-5 succeed

Producer retries msg1 AFTER msg2-5 are committed

Broker log: [msg2][msg3][msg4][msg5][msg1-retry]

ORDER CORRUPTED — msg1 is now AFTER msg2-5


FIX — Idempotent producer:

config.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);

// Assigns: ProducerID=P1, SequenceNumber=N to each message

// msg1 → (P1, Seq=0) msg2 → (P1, Seq=1)

// When msg1 retry arrives AFTER msg2-5:

// Broker: 'I already have Seq=0 → duplicate → IGNORE'

// Broker enforces: only accepts Seq=N+1 after Seq=N

// Result: no reordering, no duplicates, safe with 5 in-flight
```

Producer Transactions — Exactly-Once Across Topics

Use when a single customer action must atomically produce to multiple topics. Either ALL are published or NONE.

```
// Zalando: Order placed → atomically publish to 3 topics

kafkaTemplate.executeInTransaction(ops -> {

ops.send('order-placed', customerId, new OrderEvent(orderId, 'PLACED'));

ops.send('inventory-reserved', productId, new InventoryEvent(productId, qty));

ops.send('payment-initiated', customerId, new PaymentEvent(orderId, amount));

return true; // COMMIT — all 3 visible atomically

// exception thrown → ABORT — all 3 invisible to consumers
```

```
});


// What broker does internally:

// T+0ms: Producer → BEGIN TRANSACTION (TX-001)

// T+1-3ms: Messages sent, MARKED as TX-001, INVISIBLE to read_committed consumers

// T+4ms: Producer → COMMIT → broker writes COMMIT marker to each partition

// NOW: all 3 messages simultaneously visible to read_committed consumers


// Consumer MUST use:

config.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, 'read_committed');
```

"acks=all + idempotence = zero loss + no duplicates. Transactions = atomic multi-topic writes."

07 Consumers Deep Dive: Groups, Offsets, Auto vs Manual Commit

Consumer Groups — The Core Mechanism

A consumer group = set of consumers sharing the same **group.id**. Kafka guarantees each partition is read by **exactly ONE consumer per group**. Multiple groups can independently read the same topic simultaneously.

```
Topic: order-placed (6 partitions)

Consumer Group A (order-processing-service):
Consumer-1 → Partition 0, 1 (offset: 1240) owns these exclusively
Consumer-2 → Partition 2, 3 (offset: 1238)
Consumer-3 → Partition 4, 5 (offset: 1241)

Consumer Group B (analytics-service) — FULLY INDEPENDENT:
Consumer-1 → ALL Partitions 0,1,2,3,4,5 (offset: 800) lagging — fine

Both groups see ALL messages. Group A's offsets don't affect Group B.
Parallelism Rule: active = min(consumers, partitions). Extra consumers sit IDLE.
```

Auto-Commit vs Manual Commit — With Database

Auto-Commit — Why It's Dangerous

```
config.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, true);
config.put(ConsumerConfig.AUTO_COMMIT_INTERVAL_MS_CONFIG, 5000);

Timeline (auto.commit.interval.ms=5000):
T+0s: Consumer polls 100 messages (offsets 0-99)
T+5s: AUTO-COMMIT fires → Kafka records 'at offset 99'
(REGARDLESS of whether processing is done!)
T+7s: Consumer finishes writing messages 0-60 to DB
T+8s: SERVICE CRASHES
```

```
Restart: Kafka says 'you were at 99, start from 100'

BUT you only wrote 0-60 to DB! Messages 61-99 → LOST FOREVER
```

Manual Commit — The Correct Pattern

```
config.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);


@KafkaListener(topics = 'order-placed', groupId = 'order-processing-service')

public void processOrder(ConsumerRecord record,

Acknowledgment ack) {

try {

// DB WRITE HAPPENS FIRST

orderRepository.save(Order.builder()

.orderId(event.getOrderId()).status('PLACED').build());

// DB write succeeded → NOW tell Kafka we are done

ack.acknowledge();

} catch (DataAccessException e) {

// Do NOT acknowledge → Kafka redelivers on next poll

throw e;

}

}

// On crash after DB write but before ack:

// Restart → Kafka redelivers same message → DB write attempted again

// → Need idempotent DB write (upsert / check-then-insert)
```

Consumer Lag — Visual Explanation

```
Topic: order-placed, Partition 0


Kafka Log (what producers have written):

[msg0][msg1][msg2][msg3][msg4][msg5][msg6][msg7][msg8][msg9]

0 1 2 3 4 5 6 7 8 9
```

```
↑ Latest Offset = 9


Consumer's committed offset: 5 (last successfully processed)

LAG = Latest Offset (9) - Committed Offset (5) = 4 messages pending


Black Friday lag growth:

Producer rate: 10,000 msg/sec | Consumer rate: 2,000 msg/sec

Net lag growth: 8,000 msg/sec | After 1 min: 480,000 messages behind
```

Solutions for Consumer Lag

Solution	How	Caveat
Add consumers	Up to partition count (6 partitions → max 6 consumers)	Must have enough partitions first
Batch DB inserts	saveAll(500 rows) in one SQL call vs 500 individual save()	Most impactful for DB-heavy consumers
Async DB writes	CompletableFuture.allOf() — parallelize, commit after ALL complete	Must commit AFTER all complete — see below
Increase partitions	Allows more consumer instances	Irreversible — breaks key ordering during transition

Async DB Writes — The Dangerous Anti-Pattern

```
// WRONG — commits before DB write finishes

CompletableFuture.runAsync(() -> { orderRepository.save(...); }, executor);

ack.acknowledge(); // ← COMMITTED IMMEDIATELY — DB write still running!

// Crash now: Kafka committed, DB never wrote → ORDER LOST


// CORRECT — commit AFTER all async writes complete

List> futures = records.stream()

.map(r -> CompletableFuture.runAsync(

() -> orderRepository.save(toOrder(r.value())), executor))

.collect(Collectors.toList());

CompletableFuture.allOf(futures.toArray(new CompletableFuture[0]))

.get(30, TimeUnit.SECONDS); // timeout safety net
```

```
ack.acknowledge(); // ALL writes confirmed → NOW commit to Kafka
```

max.poll.interval.ms — The Slow DB Insert Trap

```
Two separate health mechanisms:

Heartbeat Thread (background, every 3s): 'I am alive'

Poll calls (your code): 'I am processing'


config: max.poll.interval.ms=300000 (5 min), max.poll.records=500


09:00:00 Consumer polls → gets 500 records

Background heartbeat: 'I am alive!' every 3s — Kafka believes it

Main thread: writing 500 records to DB (50ms each = 25 SECONDS)

09:05:00 max.poll.interval.ms EXCEEDED

Kafka: 'consumer has not called poll() in 5 min — STUCK!'

→ Consumer KICKED OUT → REBALANCE triggered

→ Partition reassigned to another consumer

09:05:30 Original consumer finishes all 500 DB writes ✓

Calls ack.acknowledge() → Kafka: 'you are not in the group!'

09:05:30 New consumer starts at UNCOMMITTED offset → reads same 500 → DUPLICATES!


FIXES:

Fix 1: Reduce max.poll.records to 50 (50 × 50ms = 2.5s, well within 5min)

Fix 2: Batch DB inserts — saveAll() instead of 500 individual save()

Fix 3: Async + manual commit (parallelize, commit after allOf completes)

Fix 4 (bad): Increase max.poll.interval.ms — hides the problem
```

"Manual commit is the only safe choice. Commit AFTER the DB write confirms. Make DB writes idempotent for at-least-once safety."

08 Threading Model & Timing Parameters

Default Spring Kafka Threading

```
@Bean

public ConcurrentKafkaListenerContainerFactory factory() {

ConcurrentKafkaListenerContainerFactory factory = new ...();

factory.setConsumerFactory(consumerFactory());

factory.setConcurrency(6); // 6 threads for 6 partitions

return factory;

}


concurrency=6 with 6 partitions → 6 consumer threads, each owns 1 partition

concurrency=10 with 6 partitions → only 6 active, 4 sit IDLE (wasted)

Rule: One consumer thread per partition — enforced by Kafka protocol
```

Key Threading Rules

Rule	Why
Ordering guarantee per partition	All messages in a partition are processed sequentially by ONE thread
Parallel processing within a partition	Use internal thread pool — but commit offset ONLY AFTER all complete
Offset commit ownership	Never commit from a different thread without synchronization
Heartbeat thread is automatic	Runs in background; separate from your poll/processing thread

Three Timing Parameters — The Most Confused Kafka Settings

Config	Default	What It Controls	Triggers When
heartbeat.interval.ms	3000	How often consumer sends 'I am alive' heartbeat	Every 3 seconds automatically
session.timeout.ms	45000	How long broker waits for heartbeat before declaring consumer DEAD	Heartbeat thread dies (process crash, OOM)
max.poll.interval.ms	300000	Max time between poll() calls before consumer kicked out	Consumer alive but STUCK processing (slow DB, heavy computation)

■ Critical Interview Distinction

- session.timeout.ms fires when the PROCESS CRASHES (heartbeat thread dies).
- max.poll.interval.ms fires when process is ALIVE BUT STUCK (slow DB writes, GC pause).
- They are independent checks for different failure modes — know both!

"Heartbeat = 'I am alive'. Poll interval = 'I am making progress'. Two separate checks for two different failure modes."

09 Ordering Guarantees — The Golden Rule

■ THE GOLDEN RULE

- Kafka guarantees ordering ONLY within a single partition. NEVER across partitions.
- The key you choose determines which partition receives the message.
- Therefore: the key choice determines ordering.

Zalando Order State Machine — Why Keys Matter

```
Customer CUST-101 order lifecycle:

09:00 ORDER_PLACED → hash('CUST-101') % 6 = Partition 4 → Consumer-C

09:01 PAYMENT_DONE → hash('CUST-101') % 6 = Partition 4 → Consumer-C

09:05 ITEM_PACKED → hash('CUST-101') % 6 = Partition 4 → Consumer-C

09:10 SHIPPED → hash('CUST-101') % 6 = Partition 4 → Consumer-C

09:15 DELIVERED → hash('CUST-101') % 6 = Partition 4 → Consumer-C


Consumer-C reads Partition 4 in STRICT ORDER.

State machine: ORDER_PLACED → PAYMENT_DONE → ITEM_PACKED → SHIPPED → DELIVERED


If SHIPPED arrives before PAYMENT_DONE → state machine BREAKS → wrong order status

The partition key = the ONLY thing preventing this corruption
```

Key Selection — Good vs Bad

System	Good Key	Why	Bad Key — Why Bad
E-commerce	order_id	All order status events stay sequential per order	user_id — orders are independent
Banking	account_id	Debit/credit must be applied in right sequence	transaction_type — hot partition for PAYMENT
Ride-sharing	driver_id	Location pings per driver must be sequential	city — 80% rides in NYC → hot partition
Ad clicks	ad_id	Click counts per ad need sequential processing	country — massive skew if US dominates

Ordering with Retries — Idempotent Producer Fix

```
PROBLEM (max.in.flight=5, retries=3, no idempotence):

5 messages in flight: msg1, msg2, msg3, msg4, msg5

Network error on msg1 → msg2-5 succeed → msg1 retried AFTER them

Broker log: [msg2][msg3][msg4][msg5][msg1-retry] → ORDER CORRUPTED


FIX:

config.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);

// Uses (ProducerID, SequenceNumber) to deduplicate and enforce order

// Broker rejects Seq=0 if Seq=1,2,3,4 already written → duplicate ignored

// Safe: allows 5 in-flight, maintains ordering, deduplicates retries
```

"Same key = same partition = same consumer thread = guaranteed order. This is the entire ordering story."

10 Consumer Group Rebalancing: Eager vs CooperativeSticky

Why Rebalancing Happens

A partition can only be consumed by ONE consumer per group (to maintain ordering). When group membership changes — a consumer joins or leaves — partition assignments must be recalculated. This is called a **rebalance**. Rolling deploys trigger it. Consumer crashes trigger it. New pods trigger it.

Eager Rebalance — Stop-the-World (The Problem)

```
INITIAL ASSIGNMENT: Pod-A → P0,1 | Pod-B → P2,3 | Pod-C → P4,5


Zalando deploys new version → Pod-B restarts:

Step 1: Pod-B shuts down → Group Coordinator notified

Step 2: STOP THE WORLD → ALL consumers pause consumption

Step 3: All assignments voided — Pod-A and Pod-C surrender partitions

Step 4: All consumers rejoin group, submit ownership claims

Step 5: Coordinator recalculates: Pod-A → P0,1,2 | Pod-C → P3,4,5

Step 6: Pod-A and Pod-C resume consuming

GAP: 10-30 seconds of ZERO consumption


Rolling deploy of 10 pods = 10 rebalances = up to 5 MINUTES of downtime!
```

CooperativeSticky — Incremental Rebalance (The Fix)

```
config.put(ConsumerConfig.PARTITION_ASSIGNMENT_STRATEGY_CONFIG,

List.of(CooperativeStickyAssignor.class));


Pod-B leaves:

→ Only Pod-B's partitions (2, 3) are reassigned

→ Pod-A and Pod-C KEEP their partitions and KEEP CONSUMING

→ Only brief pause for the 2 moved partitions
```

```
Rolling deploy of 10 pods:

→ Only tiny pause per pod for ~2 partitions each time

→ 98% of partitions NEVER stop consuming

→ Total downtime: ~20-40 seconds vs 5+ minutes
```

Strategy	Behaviour	Pause Duration	Use When
Range (old default)	Assigns consecutive partitions	Full stop-the-world	Small clusters only
RoundRobin	Evenly distributes across all consumers	Full stop-the-world	Balanced load needed
Sticky	Preserves existing assignments where possible	Full pause but less movement	Stateful consumers
CooperativeSticky	Incremental: only moves what needs to move	Minimal — only moved partitions pause	PRODUCTION — Zalando standard

"Eager rebalance = stop the world. CooperativeSticky = only move what must move. Always use CooperativeSticky in production."

11 Delivery Guarantees & Exactly-Once Semantics

Guarantee	Config	Data Loss?	Duplicates?	Use Case
At Most Once	acks=0, no retry, auto.commit=true	Possible	Never	Metrics, analytics, clickstream
At Least Once	acks=all + retries	Never	Possible	Most services (need idempotent consumer)
Exactly Once	Transactions + EOS + isolation.level=read_committed	Never	Never	Payments, inventory, financial

Exactly-Once via Transactions — Zalando Payment

```
// config

config.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, 'order-producer-1');

config.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);


producer.initTransactions();

try {

producer.beginTransaction();

producer.send(new ProducerRecord<>('order-placed', key, orderEvent));

producer.send(new ProducerRecord<>('inventory-reserved', key, invEvent));

producer.send(new ProducerRecord<>('payment-initiated', key, payEvent));

producer.commitTransaction(); // atomic: all 3 or none

} catch (Exception e) {

producer.abortTransaction(); // rolls back all 3

}
```

Consume-Transform-Produce — Exactly-Once Pipeline

```
// Read from topic A, process, write to topic B — exactly once

// 1. Consumer: isolation.level=read_committed

// 2. Producer: enable.idempotence=true, transactional.id set
```

```
// 3. Processing loop:

producer.beginTransaction();

// process the record...

producer.send(new ProducerRecord<>('topic-b', key, transformed));

// Atomically commit output AND input offset in one transaction:

producer.sendOffsetsToTransaction(offsets, consumerGroupId);

producer.commitTransaction();

// Either BOTH the output message and input offset advance, or NEITHER.

// This is how Kafka Streams achieves exactly-once internally.
```

Idempotent Consumer — Preventing Duplicate DB Writes

```
// When using at-least-once, consumer may receive same message twice.

// DB write must be idempotent — safe to run twice.


// Option 1: Upsert (INSERT ON CONFLICT UPDATE)

orderRepository.save(order); // Spring Data JPA upserts if entity has @Id


// Option 2: Check-then-insert

if (!orderRepository.existsByOrderId(event.getOrderId())) {

orderRepository.save(order);

}


// Option 3: Store Kafka metadata as unique constraint

// Store (topic, partition, offset) as idempotency key in DB

// Duplicate message → unique constraint violation → skip
```

"At-least-once + idempotent consumer = practically exactly-once. True EOS needs transactions but adds complexity."

12 Dead Letter Queue (DLT) — Spring Boot Patterns

What Is a Dead Letter Topic?

A Dead Letter Topic (DLT) is a regular Kafka topic where messages go after exhausting all retries. Nothing technically special — it's a naming convention. The pattern prevents **one broken message from blocking an entire partition**.

```
Normal flow:

[returns-processing] → returns-service → DB ✓


Broken message flow:

[returns-processing] → returns-service → FAILS

→ retry 1 (1s later) → FAILS

→ retry 2 (2s later) → FAILS

→ retry 3 (4s later) → FAILS

→ [returns-processing.DLT] ← parked for manual investigation

→ MAIN consumer continues with next message immediately
```

Spring Boot DLT Configuration

```
@Bean

public DefaultErrorHandler errorHandler(KafkaTemplate template) {

DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(

template,

(record, ex) -> new TopicPartition(record.topic() + '.DLT',

record.partition()));

ExponentialBackOffWithMaxRetries backOff = new ExponentialBackOffWithMaxRetries(3);

backOff.setInitialInterval(1_000L); // 1 second

backOff.setMultiplier(2.0); // then 2s, then 4s

backOff.setMaxInterval(10_000L); // cap at 10s

return new DefaultErrorHandler(recoverer, backOff);
```

```
}
```

DLT Message Headers — Automatic Metadata

```
// Kafka automatically adds to DLT messages:

kafka_dlt-exception-message: 'NullPointerException: return.sku is null'

kafka_dlt-exception-stacktrace: [full stack trace]

kafka_dlt-original-topic: 'returns-processing'

kafka_dlt-original-partition: '2'

kafka_dlt-original-offset: '4521'

kafka_dlt-original-timestamp: '2025-02-25T14:32:00Z'
```

Replaying From the DLT

```
@KafkaListener(topics = 'returns-processing.DLT', groupId = 'dlt-replay-tool')

public void replayMessage(ConsumerRecord dltRecord) {

ReturnEvent broken = dltRecord.value();

// Fix the data

if (broken.getSku() == null) {

broken.setSku(lookupSkuFromOrderId(broken.getOrderId()));

}

// Re-publish to ORIGINAL topic (not DLT)

kafkaTemplate.send('returns-processing', dltRecord.key(), broken);

}
```

■ SQS vs Kafka for DLQ

- SQS has built-in retry and DLQ support — no custom infrastructure needed.
- If primary need is reliable task queuing with automatic retries, SQS is simpler.
- Kafka excels when you need replay, multiple consumers, or high throughput.
- Naming both and explaining the trade-off signals maturity in interviews.

"DLT = parking lot for broken messages. It prevents one bad message from blocking the entire partition indefinitely."

13 Log Compaction — The Distributed KV Store Pattern

Regular Retention vs Compaction

```
REGULAR RETENTION (time-based, e.g. 7 days):

Day 8: Empty. New consumer sees NOTHING from last week.

A search indexer starting today cannot rebuild state from history.


LOG COMPACTION:

Never deletes completely. For each key, keeps ONLY the LATEST value.

New consumers get current state of ALL keys without years of history.

A new service can bootstrap current state instantly.
```

Compaction Step by Step — Zalando Product Catalog

```
product-catalog topic, Partition 0 — 6 months of history:

Offset 0: key=SKU-100 value={name:'Nike Air', price:99.99}

Offset 1: key=SKU-200 value={name:'Adidas Run',price:79.99}

Offset 2: key=SKU-100 value={price:89.99} ← price drop

Offset 3: key=SKU-300 value={name:'Puma Fly', price:59.99}

Offset 4: key=SKU-200 value={price:74.99} ← sale

Offset 5: key=SKU-100 value={price:94.99} ← price increase

Offset 6: key=SKU-200 value={price:69.99} ← further sale


AFTER COMPACTION (background cleaner runs periodically):

Offset 3: key=SKU-300 value={name:'Puma Fly', price:59.99} ← kept

Offset 5: key=SKU-100 value={price:94.99} ← latest only

Offset 6: key=SKU-200 value={price:69.99} ← latest only

Offsets 0,1,2,4 DELETED — superseded by later versions of same key
```

Why This Is Powerful — New Consumer Bootstrap

```
Zalando adds new Search Indexer service:

WITHOUT compaction:
New service reads 6 months of history
90% of work is reading stale prices overwritten later
Time to reach current state: HOURS

WITH compaction:
New service reads only LATEST price per product
Gets current state of 10 million products
Time to reach current state: MINUTES

The compacted topic acts like a PERSISTENT KEY-VALUE STORE:
SKU-100 → {name:'Nike Air', price:94.99, stock:42}
SKU-200 → {name:'Adidas Run', price:69.99, stock:8}
Any service can replay to build local cache → subscribe to stay fresh
This is exactly what Kafka Streams KTable does internally.
```

```
# Enable compaction
log.cleanup.policy=compact

# Both compaction AND time-based deletion:
log.cleanup.policy=compact,delete
log.retention.ms=86400000 # also delete after 24h

# Tombstone — how to delete from a compacted topic:
kafkaTemplate.send('product-catalog', 'SKU-999', null); // null value = delete
// After compaction: SKU-999 entry removed entirely
```

"Log compaction = keep only the latest value per key. It turns a Kafka topic into a distributed, replayable key-value store."

14 Kafka Streams — Real-Time Processing

What Is Kafka Streams?

Process, transform, and aggregate data within Kafka without external systems. Runs INSIDE your Java application — no separate cluster needed. Scales by adding partitions. State stores backed by RocksDB locally.

Real-Time Fraud Detection — Zalando (10+ uses in 5 min)

```
@Bean

public KStream fraudDetection(StreamsBuilder builder) {

KStream payments = builder.stream('payment-events');

payments

.groupByKey()

.windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(5)))

.count()

.toStream()

.filter((windowedKey, count) -> count >= 10) // 10+ in 5 min = fraud

.to('fraud-alert');

return payments;

}
```

KStream-KTable Join — Enrich Orders With Customer Profiles

```
// customer-profiles = compacted topic (acts as KV store)

// orders-stream = live order events


KTable profiles =

builder.table('customer-profiles'); // materialized to RocksDB


KStream enriched = ordersStream

.join(profiles, (order, profile) -> enrich(order, profile));


enriched.to('enriched-orders');
```

```
// During rebalance: new instance restores state store from changelog topic

// With num.standby.replicas=1: warm standby exists → near-instant restore
```

"Kafka Streams = processing inside your app. KTable = a continuously updated snapshot from a compacted topic. KStream = the live event flow."

15 Queue vs Stream: When to Use Which

The underlying Kafka mechanism is **identical** in both modes. The difference is configuration and usage pattern. Core question: **does the work need to be done once, or do multiple things need to react to the same event?**

Message Queue Use Cases

Async Processing — YouTube Video Upload

Make the standard definition video available immediately (fast path). Push the S3 URL to a Kafka topic for transcoding workers to process asynchronously. WHY: without Kafka you block the user, or lose the job if the server crashes mid-upload.

Ordered Processing — Ticketmaster Waiting Room

Users must be served in the EXACT order they arrived. A Kafka partition enforces this naturally via sequential offsets. WHY: doing this with a database requires complex row-level locking at massive scale.

Service Decoupling — Microservices Buffer

Producer generates work faster than consumer handles it. Kafka buffers it safely. Downstream service crashes? Messages accumulate. On restart, picks up exactly where it left off. WHY: one service cannot take down another. They scale independently.

Stream Use Cases

Real-Time Processing — Ad Click Aggregator

Running totals updating AS clicks happen, not hourly batches. Consumer sits on an infinite feed — it never 'finishes'. WHY: streaming enables sub-second decision making; batch processing enables SLAs measured in hours.

Multiple Independent Consumers — Facebook Live Comments

One comment event triggers moderation, fan-out, analytics simultaneously. Each is a separate consumer group reading ALL the same data independently. WHY: adding a new reaction (translation service) requires ZERO changes to existing consumers.

Hear this keyword →	Reach for →
"async", "background job", "decouple", "order matters", "queue"	Message Queue mode
"real-time", "multiple services react", "live dashboard", "event-driven", "pub/sub"	Stream mode

16 Scalability, Capacity Numbers & Hot Partitions

Capacity Numbers — Know These Cold

What	Number	Why it matters in interviews
Recommended max message size	< 1 MB	Above this: memory pressure, GC pauses, slow replication. Large payloads → S3 + URL in Kafka
Storage per broker (commodity HW)	~1 TB	RF=3 means 3TB physical storage for 1TB actual data. First scaling signal.
Throughput per broker	~1M msg/sec	Highly variable. Most interview systems 10-100x below this. Know before over-engineering.
Default retention	7 days	Configurable via retention.ms and retention.bytes. Long = more replay, more disk cost.
Typical replication factor	3 (1L+2F)	Survives 2 broker failures with acks=all. Standard production setting.

```
Quick throughput sanity check:

(daily messages) / 86,400 = avg msg/sec

Peak ≈ 5x average — use this as your design throughput


< 100K msg/sec → Single broker. Don't over-engineer.

100K-1M msg/sec → Discuss partitioning strategy.

> 1M msg/sec → Multi-broker cluster clearly needed.
```

Hot Partitions — Interviewers Love This

When key distribution is uneven, one partition receives disproportionate traffic. That broker gets overwhelmed while others sit idle. **Adding more brokers does nothing — the problem is one partition, not total cluster capacity.**

```
Example: Ad Click Aggregator partitioned by ad_id

Nike launches LeBron James ad — millions of clicks


Partition 'nike-lebron-ad': ■■■■■■■■■■■■■■■■■■■■ overwhelmed

Partition 'generic-ad-001': ■■ light

Partition 'generic-ad-002': ■ light
```

```
Adding 10 more brokers doesn't help — hot partition still on same broker.
```

Four Fixes — In Order of Preference

1. High-Cardinality Key (Preferred)

Use a key with many unique values that distributes evenly. UUIDs, snowflake IDs, user_id combos are almost always better than categorical keys (country, status). Prevention beats cure — choose the right key UPFRONT.

2. Key Salting (Random Suffix)

Append random number to the hot key: 'nike-lebron-3' instead of 'nike-lebron'. Distributes load but BREAKS per-ad ordering — all clicks for that ad are no longer sequential. Acceptable when you only need aggregate counts, not strict order.

3. Compound Key

Combine ad_id with another dimension: 'ad_id:region' or 'ad_id:user_bucket'. Spreads traffic while keeping partial locality. Useful when partial ordering suffices.

4. Back Pressure

Slow down the producer when partition lag exceeds threshold. Doesn't fix distribution — prevents system collapse. A safety valve, not a primary solution.

■■ Scaling Strategies

- Add more brokers: distributes partitions, increases total capacity. Need enough partitions to use them.
- Add more partitions: increases parallelism. Hard to reduce later — slightly over-partition upfront.
- Managed services (Confluent Cloud, AWS MSK): auto-scaling handled. Mention this to show pragmatism.

"Hot partitions come from low-cardinality keys. Fix: better key, not more hardware."

17 Performance Optimisations

Producer Optimisation Table

Config	Direction	Value	Impact
linger.ms	Increase	20-50ms	Larger batches, higher throughput (+10x for small messages)
batch.size	Increase	64KB-128KB	Fewer network calls per unit of data
compression.type	Set	snappy / zstd	40-75% less network bandwidth and storage
acks	Lower for throughput	0 or 1	Higher throughput, less durability — use only for metrics/analytics
enable.idempotence	Enable	true	Safe retries + ordering + 5 in-flight — use always for important topics
buffer.memory	Increase	64-128MB	Handles more in-flight without blocking producer

Consumer Optimisation Table

Config	Direction	Value	Impact
fetch.min.bytes	Increase	1MB	Fewer fetch requests, higher throughput
fetch.max.wait.ms	Adjust	500ms	Wait for min.bytes before returning
max.poll.records	Reduce if slow	50-100	Prevent max.poll.interval breach on slow DB
max.poll.records	Increase if fast	1000	Larger batches per poll for fast consumers
enable.auto.commit	Disable	false	Manual control — prevents data loss
session.timeout.ms	Increase	30-45s	Avoid false rebalances on slow consumers

Topic & Cluster Optimisation

Config	Recommended	Reason
replication.factor	3	Survive 2 broker failures
min.insync.replicas	2	Durability without blocking on 1 slow broker
num.partitions	= max expected consumers	Maximum parallelism ceiling — over-provision slightly
log.retention.ms	604800000 (7 days)	Enough for replay, not too much storage

log.cleanup.policy	compact (state topics)	KTable / changelog topics need compaction for new service bootstrap
unclean.leader.election	false	No data loss over availability — always for financial data

18 Complete Configuration Reference

Configuration Key	Default	What It Controls
replication.factor	1	Number of copies of each partition across brokers
min.insync.replicas	1	Min ISR count required for acks=all writes
acks	1	Producer durability: 0=none, 1=leader, all=ISR
unclean.leader.election	false	Allow out-of-sync replicas to become leader (risk: data loss)
replica.lag.time.max.ms	30000	Max ms follower can lag before removed from ISR
log.retention.ms	604800000	How long to retain messages (default 7 days)
num.partitions	1	Default partition count for new topics
auto.offset.reset	latest	Where consumer starts when no committed offset exists
enable.auto.commit	true	Consumer auto-commits offsets periodically — DISABLE in production
session.timeout.ms	45000	Max ms before broker declares consumer dead (heartbeat)
heartbeat.interval.ms	3000	How often consumer sends alive heartbeat
max.poll.interval.ms	300000	Max ms between poll() calls before consumer kicked out
max.poll.records	500	Max records returned per poll call
enable.idempotence	false	Enable exactly-once producer semantics — use always for important topics
linger.ms	0	Wait time to fill batches before sending
batch.size	16384	Max batch size in bytes (16KB default — increase to 64-128KB)
compression.type	none	Message compression: none, gzip, snappy, lz4, zstd
log.cleanup.policy	delete	Retention strategy: delete or compact
fetch.min.bytes	1	Minimum bytes for consumer fetch response
fetch.max.wait.ms	500	Max wait for fetch.min.bytes to be met
partition.assignment.strategy	Range	Consumer partition assignment: CooperativeStickyAssignor for production
isolation.level	read_uncommitted	Consumer: read_committed for exactly-once
transactional.id	null	Unique producer ID for exactly-once transactions

`num.standby.replicas`	0	Kafka Streams: warm standby for fast state restore on rebalance

19 30 Hard Interview Q&A; — Zalando / FAANG Level

Framework: P-A-R-T — Partitioning · Acknowledgment · Reliability · Throughput

ORDERING & PARTITIONING

Q1. You have a 10-partition order topic. Orders from the same customer must be processed in order, but VIP customers need priority. How do you design this?

Separate VIP and standard into two topics: 'order-placed-vip' (2 partitions, dedicated consumer group with more threads) and 'order-placed-standard' (8 partitions). Both use customerId as key for per-customer ordering. VIP consumers run with higher resource limits and separate auto-scaling. Alternatively, encode priority in the message header and use a routing consumer — but this breaks Kafka parallelism and complicates backpressure.

Q2. Your team increases partition count from 10 to 20 on order-placed. What breaks?

Key-based routing breaks. hash(key) % 10 and hash(key) % 20 produce different results. Customers previously on partition 3 may now land on partition 13. Consumers processing partition 3's history won't see new messages for those customers. In-flight messages during transition can be mis-routed. Fix: dual-consume old+new topics during migration, use round-robin (keyless) produces during transition, or use consistent hashing instead of modulo.

Q3. After a crash and restart, your consumer reprocesses 1000 already-handled messages. Why, and how do you prevent it without losing messages?

Consumer wrote to DB but crashed before calling ack.acknowledge(). On restart, Kafka redelivers from last committed offset. Prevention: make DB writes idempotent (upsert with orderId as unique key, or check-then-insert). Never use manual ack without idempotent consumers for at-least-once delivery. For exactly-once: use transactions — requires entire pipeline to support EOS.

Q4. With max.in.flight=5 and retries=3, explain how message reordering happens and how idempotent producer prevents it.

5 messages in flight simultaneously. Network error on msg1 only; msg2-5 succeed and are written. msg1 retry arrives after msg2-5, landing later in log: [msg2][msg3][msg4][msg5][msg1-retry] — ordering corrupted. Idempotent producer assigns (ProducerID, SequenceNumber). Broker enforces: only accepts Seq=N+1 after Seq=N. Late msg1 retry with Seq=0 is recognized as duplicate and rejected. Result: ordering maintained safely with 5 in-flight.

Q5. 6 regional warehouses produce inventory events. You need global inventory view ordered by SKU. Design the partition strategy.

Use SKU as partition key across a single topic 'inventory-events'. All 6 warehouses publish using skuId as key — same SKU always lands in same partition regardless of warehouse. Ordering guarantee: all inventory changes for SKU-100 are sequentially processed by one consumer. Use Kafka Streams KTable with changelog topic to maintain current inventory count per SKU. Trade-off: hot SKUs create partition hot spots — mitigate with compound key (skuId + suffix for very high-frequency SKUs).

CONSUMER GROUPS & REBALANCING

Q6. Black Friday: 20 consumers, 20 partitions, rolling deploy. Each EagerRebalance takes 30s. Calculate total downtime. How does CooperativeSticky fix this?

20 pod restarts × 30 seconds per rebalance = 600 seconds = 10 minutes of total consumption downtime. With EagerRebalance, every single restart triggers full group rebalance where ALL consumers stop. With CooperativeSticky: only the 1-2 partitions from the dying pod are reassigned. Other 19 consumers keep consuming uninterrupted. Total downtime: ~1-2s per pod × 20 = 20-40 seconds total. Config: partition.assignment.strategy = CooperativeStickyAssignor.

Q7. Consumer-3 is processing a heavy message taking 8 minutes with max.poll.interval.ms=300000. What happens?

max.poll.interval.ms (5 minutes) is exceeded. Kafka's group coordinator declares Consumer-3 as stuck (not dead — heartbeat thread may still be alive). Consumer-3 is forcibly removed from the group, triggering rebalance. Its partitions are reassigned. Consumer-3 eventually finishes, tries to commit offset, receives CommitFailedException. Message will be reprocessed by the new partition owner. Fix: reduce max.poll.records to process fewer messages per poll, or split heavy processing into a separate thread pool.

Q8. Fraud checker needs low latency (can miss messages). Audit logger must never miss. Same topic. How?

Both use different consumer group IDs — consume independently. Fraud checker: enable.auto.commit=true, auto.offset.reset=latest, max.poll.records=1. Audit logger: enable.auto.commit=false, auto.offset.reset=earliest, manual commit after DB write, DLT configured. The fraud checker's lag doesn't affect the audit logger since they maintain separate offset pointers.

Q9. Producer sends 50,000 msg/sec, consumer processes 10,000 msg/sec. Lag grows. Options?

(1) Scale consumers — add instances up to partition count. 6 partitions × 10K = 60K msg/sec — solves it. (2) Optimize consumer — batch DB inserts (saveAll vs save), async processing, connection pooling. (3) Increase partitions — irreversible, breaks key-ordering during transition. (4) Rate limit producer. (5) Increase retention — won't fix lag but prevents data loss while you solve root cause. Always monitor lag via Prometheus + Grafana before choosing.

Q10. Explain session.timeout.ms vs heartbeat.interval.ms vs max.poll.interval.ms. Which triggers for a slow-but-alive consumer?

heartbeat.interval.ms (3s): how often background thread pings broker. session.timeout.ms (45s): if no heartbeat received within this window, broker declares consumer DEAD. max.poll.interval.ms (300s): if main thread doesn't call poll() within this window, consumer declared STUCK and kicked out. A slow-but-alive consumer (heartbeat thread running, but main thread blocked on slow DB) triggers max.poll.interval.ms, NOT session.timeout.ms. They are independent checks for different failure modes.

EXACTLY-ONCE & TRANSACTIONS

Q11. Atomically: (a) deduct inventory, (b) charge payment via REST API, (c) publish order confirmation. How?

Pure Kafka transactions only cover Kafka-to-Kafka. For REST API involvement, use Saga pattern with compensating transactions: (1) Publish ORDER_STARTED to Kafka, (2) Inventory service consumes, deducts, publishes INVENTORY_CONFIRMED, (3) Payment service calls REST API, on success publishes PAYMENT_CONFIRMED; on failure publishes PAYMENT_FAILED, (4) If payment fails: publish INVENTORY_ROLLBACK. Each step idempotent with correlationId. Alternatively: Outbox pattern — write to DB and Kafka atomically using DB transaction, then CDC relay (Debezium) publishes to Kafka.

Q12. You restart your producer with a new transactional.id. What happens to in-flight transactions?

The broker uses transactional.id to maintain transaction state across restarts. When a producer with same transactional.id starts, it calls initTransactions() which performs 'zombie fencing' — bumps the producer epoch. Any in-flight transactions from the previous instance (old epoch) are automatically aborted when broker sees messages from the higher epoch. This prevents the old zombie producer from committing stale transactions after recovery.

Q13. You read from topic A, process, write to topic B. How do you achieve exactly-once for the whole pipeline?

Use Kafka's consume-transform-produce (CTP) transaction: (1) Consumer with isolation.level=read_committed. (2) Producer with enable.idempotence=true and transactional.id. (3) In processing loop: beginTransaction(), process record, produce to topic B, then sendOffsetsToTransaction(offsets, consumerGroupId) — atomically commits consumer offset AND produced message. Either both advance or neither. This is exactly how Kafka Streams achieves exactly-once internally.

Q14. You use acks=1. Leader crashes after ACK but before replication. How to detect and handle?

You cannot detect this from the producer side — the ACK was already sent and the message is permanently lost. The new leader (promoted follower) doesn't have it. Prevention: switch to acks=all with min.insync.replicas=2. To detect after the fact: implement sequence numbers or correlation IDs in messages; downstream consumers detect gaps. Build a reconciliation job comparing source system counts with Kafka message counts per time window.

Q15. Why can't Kafka transactions span external databases natively? What patterns work?

Kafka transactions use a distributed commit protocol internal to Kafka. External databases have their own transaction managers. Cross-system 2PC would require XA transactions — slow, complex, not supported by most Kafka clients. Alternatives: (1) Outbox Pattern: write to DB and outbox table in same DB transaction; CDC (Debezium) reads outbox and publishes to Kafka. (2) Event Sourcing: Kafka is source of truth; DB is derived. (3) Saga with compensating transactions. (4) Spring @Transactional + KafkaTransactionManager for simple cases.

PERFORMANCE & SCALABILITY

Q16. Zalando processes 1 million order events per minute. Design the Kafka topic configuration.

1M/min = ~16,666/sec. Single partition handles ~50K/sec, so 1 partition is technically enough but use 20 for consumer parallelism. RF=3, min.insync.replicas=2. Retention: 7 days (604800000ms). Compression: zstd. acks=all. linger.ms=10, batch.size=128KB. Disable auto.create.topics. Key: customerId for ordering. Monitor with Prometheus + Grafana. Mention Confluent Cloud / MSK to show pragmatism.

Q17. linger.ms=50, batch.size=128KB, sending 100-byte messages. Throughput vs linger.ms=0? When NOT to batch?

With linger.ms=50 at 10K msg/sec: in 50ms accumulate 500 messages × 100 bytes = 50KB batch. One network call per 50ms = 20 batches/sec. Without batching: 10,000 individual calls/sec. Improvement: 500x fewer calls, ~10x better throughput. Do NOT batch when: (1) Latency critical and 50ms delay unacceptable (live trading, real-time alerting), (2) Message rate very low (1 msg/min — batching just adds latency), (3) Single message contains large critical event needing immediate delivery.

Q18. 100 partitions, 10 brokers. One broker handles 70 partition leaders due to skewed keys. How to detect and fix?

Detection: (1) Monitor NetworkProcessorAvgIdlePercent and MessagesInPerSec JMX metrics per broker, (2) kafka-topics.sh --describe shows leader distribution, (3) Grafana bytes-in per broker dashboard. Fixes: (1) kafka-preferred-replica-election.sh redistributes leaders, (2) kafka-reassign-partitions.sh manually rebalances, (3) auto.leader.rebalance.enable=true (default), (4) Root cause: change partition key to higher cardinality (customerId + orderId compound key).

Q19. Process Kafka messages, write to PostgreSQL (50ms per write), 6 partitions. Max throughput? How to increase without breaking ordering?

6 consumer threads × (1000ms / 50ms) = 120 messages/sec max. Increase WITHOUT breaking ordering: (1) Async DB writes within a partition — CompletableFuture.allOf() to parallelize DB writes for messages in same batch, commit offset only after all complete. Maintains partition ordering while allowing concurrent writes within a batch. (2) Batch inserts: 500 rows in one saveAll() takes ~60ms instead of 25,000ms. Throughput: 6 × (500/60ms) = ~50K msg/sec. (3) Increase DB connection pool size.

Q20. Compare gzip vs snappy vs lz4 vs zstd for high-volume small JSON order events.

Small JSON (100-500 bytes) at high volume: gzip — 70%+ savings but 5x CPU of snappy. For 1M events/sec, CPU cost significant. snappy — 60% savings, low CPU, fast decompress. Good general choice. lz4 — 40% savings, lowest CPU, fastest compress+decompress. Best for latency-sensitive paths. zstd — 75% savings, moderate CPU, tunable levels. Best overall: zstd for archival/analytics topics, lz4 for real-time order events where latency matters more than ratio.

FAULT TOLERANCE & RELIABILITY

Q21. 3-broker cluster, min.insync.replicas=2, RF=3, one broker down. What's affected? What if min.insync=3?

With min.insync.replicas=2 and 1 broker down: ISR shrinks from [B1,B2,B3] to [B2,B3]. ISR size=2 >= min.insync=2. Reads and writes continue normally. Zero impact. With min.insync.replicas=3 and 1 broker down: ISR size=2 < min.insync=3. Producers with acks=all receive NotEnoughReplicasException. Writes REJECTED until third broker recovers. Reads still work. This is consistency-over-availability — appropriate only for data where partial writes are worse than unavailability.

Q22. A poison pill message causes NullPointerException, blocking an entire partition. Design robust error handling.

(1) Wrap consumer logic in try-catch per message. (2) Retry with exponential backoff (1s, 2s, 4s) using DefaultErrorHandler. (3) After max retries, route to DLT via DeadLetterPublishingRecoverer — partition continues with next message. (4) DLT listener alerts ops team with full headers. (5) Add message validation before processing (Schema Registry). (6) Circuit breaker if error rate exceeds threshold. (7) Key distinction: non-retryable exceptions (NPE, invalid data) go directly to DLT; retryable (network timeout, DB timeout) are retried.

Q23. Payment service crashes, restarts, re-reads 10,000 orders, attempts to charge customers twice. Design idempotent consumers.

(1) DB-level: store orderId as unique key in payments table. Duplicate charge attempt throws ConstraintViolationException — catch and skip. (2) Redis dedup cache: before processing, check if orderId already processed. If yes, skip. TTL = 7 days. (3) Outbox pattern: payment service writes to its own outbox DB table (idempotent upsert), separate process publishes confirmation. (4) Kafka metadata: store (topic, partition, offset) as idempotency key with unique constraint. Duplicate → skip.

Q24. Replay all messages from last 24 hours for a new analytics service without affecting existing consumer groups.

Create a new consumer group with unique group.id: 'analytics-replay-service'. Use kafka-consumer-groups.sh --reset-offsets --to-datetime 2025-02-24T00:00:00 --group analytics-replay-service --topic order-placed. Or in Spring: KafkaConsumerSeekAware interface with seekToTimestamp() method. The existing consumer groups are completely unaffected — they have their own committed offsets. This is Kafka's key advantage over traditional queues.

Q25. Design a Saga for Zalando order flow (Order → Payment → Inventory → Shipping). Handle compensating transaction when Inventory fails after Payment succeeded.

Choreography Saga. Events: order-started, payment-completed, payment-failed, inventory-reserved, inventory-failed, order-cancelled, shipping-scheduled. Compensation flow: Inventory publishes inventory-failed → Payment service listens, issues refund, publishes payment-refunded → Order service marks cancelled, publishes order-cancelled. Each compensation is idempotent (retryable). Saga state tracked via Saga Orchestrator or event sourcing. Use correlationId (orderId) as key for ALL events — ensures they land in same partition for ordering.

ADVANCED DESIGN

Q26. Design real-time recommendation system: 100K user events/sec, update product affinity within 2 seconds.

Kafka Streams topology: (1) Stream 'user-browse-events' (userId key). (2) Windowed aggregation: count product views per user in 30-min sliding window using SessionWindows. (3) KTable join with product-catalog compacted topic. (4) Output to 'user-affinity-scores' (userId key) with top-10 affinities. (5) Recommendation API subscribes and serves from local in-memory state store. 100K/sec → 20+ partitions. State stores local to each Streams instance; use interactive queries for cross-instance lookups. num.standby.replicas=1 for fault tolerance.

Q27. Join stream of order events with static customer profile table in Kafka Streams. What join type? How does state store work? What happens during rebalance?

Use KStream-KTable join. Profiles stored as compacted KTable topic. State store: Kafka Streams materializes KTable into RocksDB locally. Each instance holds only its partition's profiles. During rebalance: new instance must restore state stores from changelog topic. Restoration time = how far behind the standby is. With num.standby.replicas=1, warm standby exists and restoration is near-instantaneous.

Q28. GDPR right-to-erasure: customer requests deletion from Kafka's immutable log. How?

(1) Crypto-shredding: encrypt PII fields with per-user key stored in Azure Key Vault. On deletion: delete the key. Messages exist but PII is unreadable (effectively deleted). Low operational cost, elegant. (2) Null tombstone on compacted topics: publish tombstone (null value) with customerId as key. After compaction cycle, record removed. Only works for compacted topics. (3) Data masking: 'scrubber' consumer re-publishes with PII replaced by tokens. Most complex, highest confidence. Recommendation: crypto-shredding for audit/event topics, tombstones for compacted state topics.

Q29. Design multi-region Kafka for Zalando (EU + US). Cross-region order events, no duplicate order IDs, failover.

Use MirrorMaker 2 or Confluent Replicator. Active-Active: both regions accept orders, replicated to each other. Duplicate prevention: prefix order IDs with region code (EU-ORDER-001, US-ORDER-001). Topics: EU-primary topics mirrored to US as 'eu.order-placed'. Failover: health checks monitor replication lag; if EU down, US consumers switch to mirrored 'eu.*' topics. GDPR: EU customer PII topics blocked from US mirroring via topic-level ACLs.

Q30. Compare Kafka vs Kinesis vs Pulsar vs Azure Event Hubs. You're on Azure — hidden operational differences?

Self-managed Kafka: full control, highest ops burden. Azure Event Hubs with Kafka protocol: drop-in replacement, managed service. Hidden differences: partition count FIXED at creation (no changes), max 1024 partitions per namespace, retention max 90 days, no Kafka Streams support natively, limited EOS. Kinesis: max 1MB/sec per shard, max 7 days retention, no Kafka API compatibility. Pulsar: multi-tenancy built-in, native geo-replication, smaller ecosystem. Zalando on Azure: Event Hubs for most services (reduced ops), self-managed Kafka for Kafka Streams workloads or where partition flexibility matters.

20 Master Cheat Sheets

Fault Tolerance Quick-Reference

Goal	Configuration
Survive N broker failures	`replication.factor = N + 1`
Guarantee no data loss	`acks=all, min.insync.replicas=2`
No dirty reads / split-brain	`unclean.leader.election=false`
Survive consumer crash	Consumer groups auto-rebalance; commit after DB write
No duplicate messages	`enable.idempotence=true`
Exactly-once end-to-end	`Transactions + isolation.level=read_committed`
No rebalance storm on deploy	`CooperativeStickyAssignor`
Handle poison pill messages	DefaultErrorHandler + DLT + ExponentialBackOff

Producer Config Cheat Sheet

Scenario	Config	Value
Zero data loss	`acks`	all
Zero data loss	`min.insync.replicas`	2
No duplicates on retry	`enable.idempotence`	true
Max throughput	`linger.ms`	20-50ms
Max throughput	`batch.size`	65536-131072
Reduce network/storage	`compression.type`	snappy or zstd
Atomic multi-topic writes	`transactional.id`	unique-per-producer-instance
Fire and forget (metrics)	`acks`	0
Order with retries	`enable.idempotence`	true (sets max.in.flight=5 safely)

Consumer Config Cheat Sheet

Scenario	Config	Value
Never lose messages	`enable.auto.commit`	false (manual ack)

Prevent duplicate DB writes	Implement idempotent consumer	Upsert or check-then-insert
Slow DB writes	`max.poll.records`	50-100 (not 500)
Exactly-once pipeline	`isolation.level`	read_committed
Zero-downtime deploy	`partition.assignment.strategy`	CooperativeStickyAssignor
Read from beginning (new svc)	`auto.offset.reset`	earliest + new group.id
Avoid false rebalances	`session.timeout.ms`	30000-45000
Scale consumers	concurrency (Spring)	= num.partitions

Leader / ISR / HW / LEO Quick Reference

Question	Answer
Leadership is per...?	PARTITION — not per broker, not per topic
Who writes to Kafka?	ALL producers write ONLY to the partition LEADER
Who reads from Kafka?	ALL consumers read ONLY from LEADER (default)
What do followers do?	Silently replicate. NEVER serve reads or writes.
What is ISR?	Set of replicas fully caught up to leader. Only ISR can become new leader.
What is HW?	Highest offset replicated to ALL ISR members. Max visible to consumers.
What is LEO?	Log End Offset — latest offset written to THIS specific replica.
If all ISR members fail?	Partition goes OFFLINE (unclean.leader.election=false)
When does leader change?	Crash, graceful shutdown, admin reassignment, controller decision

Delivery Guarantee Decision Tree

If You Need	Use
Max throughput, some loss OK	acks=0, no retry, auto.commit=true
No loss, occasional duplicates OK	acks=all + retries + idempotent consumer
No loss, no duplicates (financial)	acks=all + enable.idempotence + transactions
Cross-topic atomicity	Kafka Transactions (executeInTransaction)
Cross-service atomicity	Saga pattern + compensating transactions
DB + Kafka atomicity	Outbox pattern + Debezium CDC

Zalando Interview Framework — P-A-R-T

Letter	Stands For	Key Questions
P	Partitioning Strategy	What key? How many partitions? What ordering needs?
A	Acknowledgment & Durability	acks setting? RF? min.insync.replicas? Can we tolerate data loss?
R	Reliability Pattern	Idempotency? Transactions? DLQ? Retry strategy? EOS?
T	Throughput Optimisation	Batching? Compression? Consumer parallelism? Async writes?

All Mental Models — Quick Revision

"Kafka is a distributed log. Everything else — queuing, streaming, replication, scaling — flows from that single idea."

"A topic is a label. A partition is a file. The label routes your data; the file stores it."

"A disk write is fast. A disk seek is slow. Append-only means you seek once — to the end — then just keep writing."

"Same key = same partition = same consumer thread = guaranteed order."

"Producer hashes key → looks up broker in local metadata → writes directly. No gateway, no routing layer."

"Leadership is per-partition. ISR = replicas fully caught up. HW = what all ISR confirmed. Consumers only read up to HW."

"Replication factor = how many broker deaths you can survive. RF=3 survives 2 failures."

"One partition → one consumer per group. Multiple groups each see everything. Offsets are bookmarks."

"Hot partitions come from low-cardinality keys. Fix: better key, not more hardware."

"If your numbers fit in one broker, scaling is not the conversation. Know the numbers first."

"At-least-once is the default. Make your consumer idempotent and it's practically exactly-once."

"Kafka is always available. 'What if Kafka goes down?' — gently push back on this question."

"Eager rebalance = stop the world. CooperativeSticky = only move what must move."

"Log compaction = keep only the latest value per key. Turns a topic into a distributed KV store."

"Manual commit + idempotent DB writes = the foundation of reliable Kafka consumption."

■ Zalando Bonus — Always Mention These

- GDPR awareness: crypto-shredding for EU PII data, tombstones for compacted topics.
- EU-scale throughput planning: estimate msg/sec before recommending partition count.
- Exactly-once for payments: acks=all + idempotence + transactions.
- CooperativeStickyAssignor: zero-downtime deploys — know this inside out.
- Schema Registry: contract enforcement between services, prevents data format breaking changes.
- Managed services: Confluent Cloud / Azure Event Hubs — show you know operational realities.

Good luck — you've got this. ■

Apache Kafka Master Reference · Zalando & FAANG Interview Preparation